

MINI-PROJECT REPORT

Packet-Level Traffic Analysis for Anomaly Detection using Wireshark

Submitted by

Anbuselvi. S

25237197207

Course Name: Artificial Intelligence for Cybersecurity

Course Code: CB3201

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING GUINDY
ANNA UNIVERSITY

May 2026

1. Objective

- To capture and analyze network packets using Wireshark and tshark.
- To detect abnormal network traffic patterns using Machine Learning techniques.
- To build a real-time anomaly detection dashboard using Streamlit.
- To identify suspicious traffic such as packet bursts, oversized packets, and abnormal traffic behavior.

2. Tool(s) Used

Tool	Purpose
Wireshark	Packet capture and protocol analysis
tshark	CLI-based packet feature extraction
Python	Data processing and ML
Pandas and NumPy	Feature engineering
Scikit-learn	Isolation Forest anomaly detection
Streamlit	Real-time dashboard
Plotly	Interactive visualization

3. Hardware Requirements

The following hardware configuration is recommended for efficient packet capturing, traffic processing, and anomaly detection.

- **Processor** : Intel Core i5 / AMD Ryzen 5 or above
- **RAM** : Minimum 8 GB
- **Storage** : At least 10 GB free disk space
- **Network Adapter** : Ethernet or WiFi Interface Card
- **Display** : Minimum 1366 × 768 resolution
- **Operating System** : Ubuntu Linux / Windows / macOS

The hardware configuration should support live packet capturing, real-time dashboard visualization, and machine learning-based anomaly detection without significant performance degradation.

4. Software Setup

This section describes the installation and configuration steps required to set up the environment for packet-level traffic analysis and anomaly detection.

Required Software

- Python 3.10 or above
- Wireshark
- tshark
- Streamlit
- Nmap

Step 1 – Install Python

Install Python 3.10 or above from the official Python website.

<https://www.python.org/downloads/>

Verify the installation using:

```
python3 --version
```

Step 2 – Install Wireshark and tshark

Install Wireshark and tshark using the following commands:

```
sudo apt update
sudo apt install wireshark tshark
```

Verify tshark installation:

```
tshark -v
```

Step 3 – Install Nmap

Install Nmap for generating scanning traffic used in anomaly detection.

```
sudo apt install nmap
```

Verify installation:

```
nmap --version
```

Step 4 – Create Python Virtual Environment

Create and activate a virtual environment for the project.

```
python3 -m venv venv
source venv/bin/activate
```

Step 5 – Install Required Python Libraries

Install all required Python libraries using pip.

```
pip install pandas numpy scikit-learn \
streamlit plotly matplotlib
```

Step 6 – Project Files Setup

Place the following files inside the project directory:

- app.py
- attack.pcapng
- features.csv
- requirements.txt

Step 7 – Run the Streamlit Dashboard

Execute the Streamlit application using:

```
streamlit run app.py
```

After execution, the dashboard will be available in the browser at:

<http://localhost:8502>

Step 8 – Verify Successful Setup

The dashboard should display:

- Traffic activity timeline
- Suspicious hosts
- Packet statistics
- ML-based anomaly detection results
- Packet size distribution graphs

5. Architecture / Flow Diagram

System Workflow

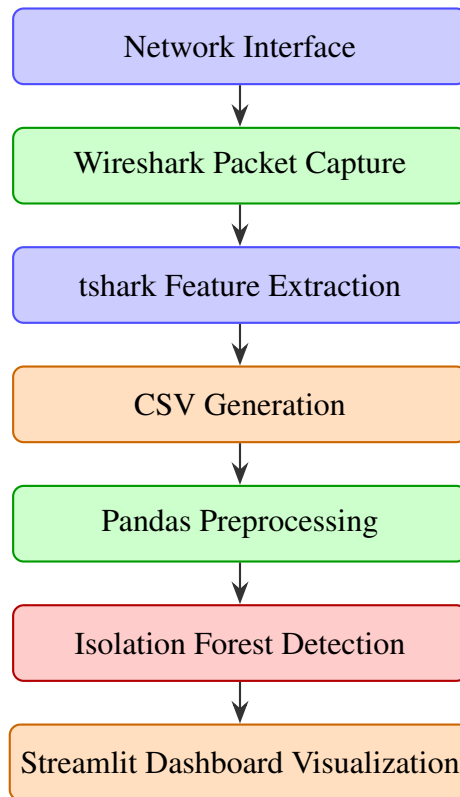


Figure 1: Packet-Level Traffic Analysis and Anomaly Detection Workflow

6. Algorithm / Working Procedure

Step 1 – Packet Capture

Wireshark captures live packets from the network interface.

Step 2 – Feature Extraction

tshark extracts packet fields into CSV format.

```
tshark -r capture.pcapng -T fields \  
-e frame.number \  
-e frame.time_relative \  
-e ip.src \  
-e ip.dst \  
-e tcp.srcport \  
-e tcp.dstport \  
-e frame.len \  
-E header=y \  
-E separator=, \  
> features.csv
```

Step 3 – Data Preprocessing

- Remove null values
- Convert timestamps
- Generate traffic features

Step 4 – ML-Based Detection

Isolation Forest detects abnormal traffic behavior.

```
model = IsolationForest(  
    n_estimators=200,  
    contamination=0.01,  
    random_state=42)
```

Step 5 – Traffic Spike Detection

Packets are resampled into 1-second bins to detect abnormal traffic bursts.

```
pps = df.resample("1s").size()
```

Step 6 – Visualization

Results are displayed using Streamlit dashboards:

- Traffic spikes
- Top talkers
- Packet size histogram
- ML anomalies
- PPS graphs

7. Use Cases

- **Enterprise SOC** – Continuous monitoring of enterprise network traffic and detection of suspicious and anomalous activities.
- **Academic Research** – Network traffic dataset generation and machine learning-based traffic analysis.
- **IoT Security** – Detection of rogue IoT devices and monitoring abnormal IoT communication.
- **Digital Forensics** – Post-incident packet analysis and reconstruction of attack timelines.
- **Education and Labs** – Cybersecurity practical training and packet-level traffic analysis learning.

8. Step-by-Step Execution with Result Screenshots

Step 1 – Capture Packets

Open Wireshark and select the active network interface such as `eth0` or `wlan0` to begin live packet capturing.

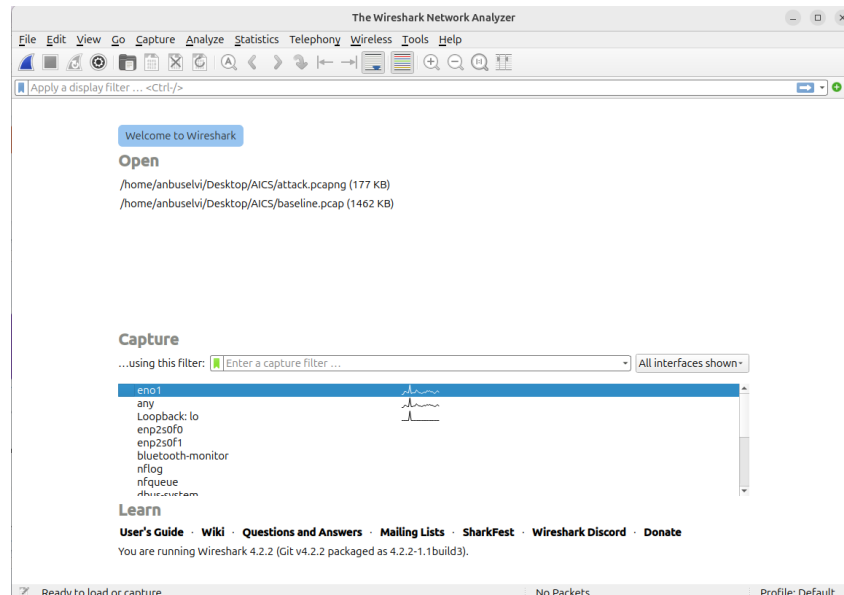


Figure 1: Wireshark Interface Selection Window

The Wireshark home screen displays all available network interfaces for capturing packets.

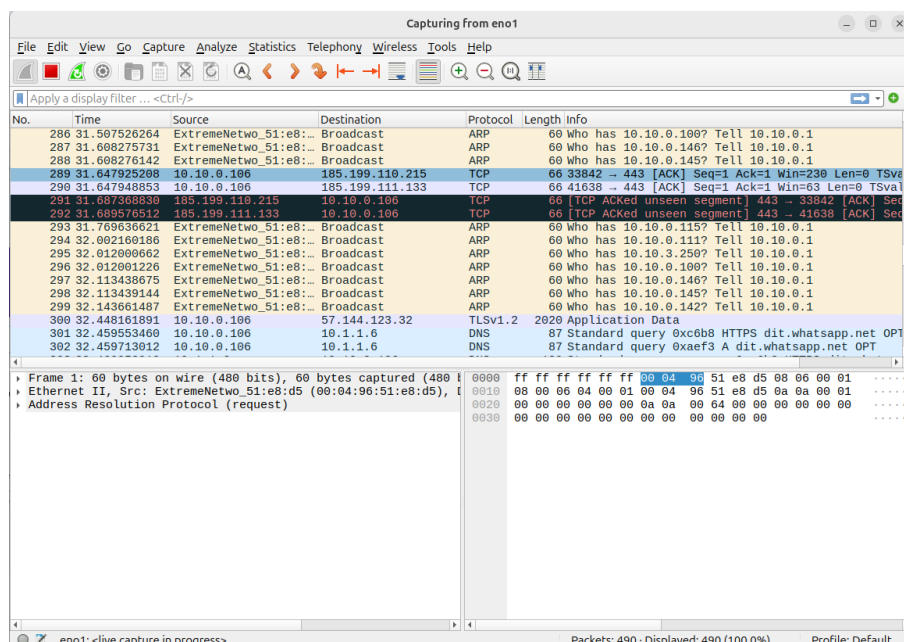
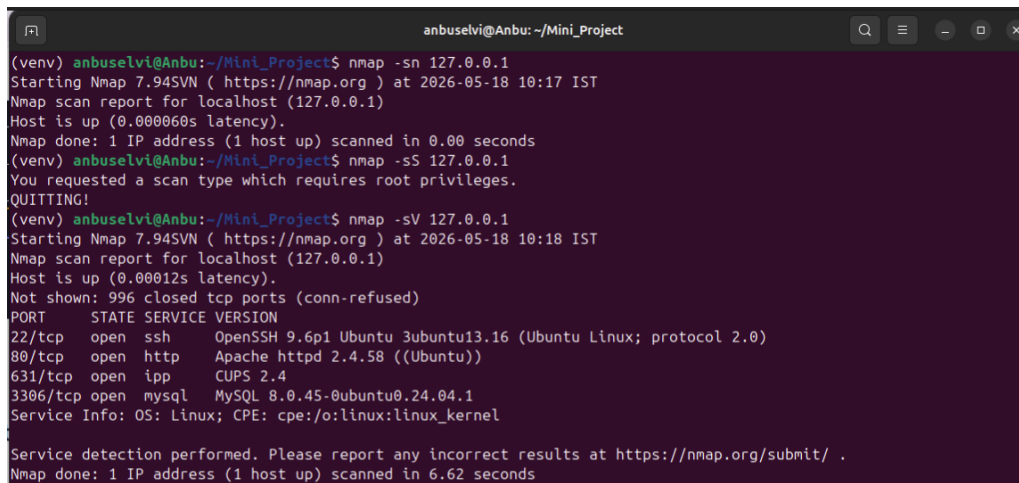


Figure 2: Live Packet Capture in Wireshark

Live network packets captured from the selected interface are displayed with protocol, source, destination, and packet details.

Step 2 – Generate Traffic

Nmap commands are executed to generate scanning and reconnaissance traffic for anomaly analysis.

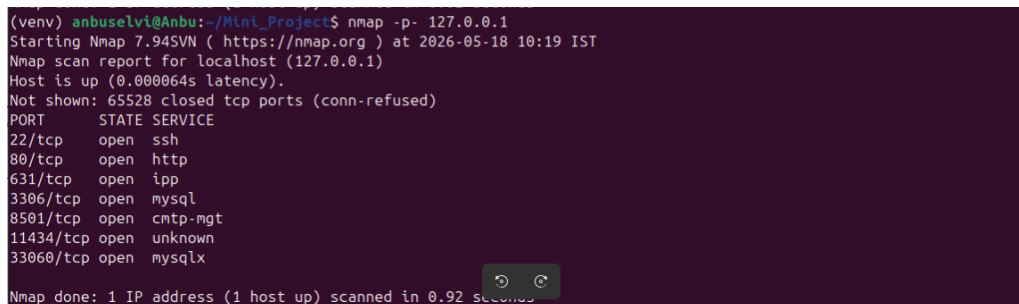
A terminal window titled 'anbuselvi@Anbu: ~/Mini_Project' showing the execution of Nmap commands. The first command is 'nmap -sn 127.0.0.1', which reports the host is up. The second command is 'nmap -sS 127.0.0.1', which is refused due to lack of root privileges. The third command is 'nmap -sV 127.0.0.1', which performs a service version scan, reporting various open ports and services like ssh, http, ipp, and mysql.

```
(venv) anbuselvi@Anbu:~/Mini_Project$ nmap -sn 127.0.0.1
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-18 10:17 IST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000060s latency).
Nmap done: 1 IP address (1 host up) scanned in 0.00 seconds
(venv) anbuselvi@Anbu:~/Mini_Project$ nmap -sS 127.0.0.1
You requested a scan type which requires root privileges.
QUITTING!
(venv) anbuselvi@Anbu:~/Mini_Project$ nmap -sV 127.0.0.1
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-18 10:18 IST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00012s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.58 ((Ubuntu))
631/tcp   open  ipp      CUPS 2.4
3306/tcp  open  mysql    MySQL 8.0.45-0ubuntu0.24.04.1
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.62 seconds
```

Figure 3: Nmap Host Discovery Scan

The host discovery scan checks whether the target system is active and reachable.

A terminal window showing the execution of 'nmap -p- 127.0.0.1'. The output lists several open ports and the services running on them, including ssh, http, ipp, mysql, cmtp-mgt, and an unknown service on port 11434.

```
(venv) anbuselvi@Anbu:~/Mini_Project$ nmap -p- 127.0.0.1
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-18 10:19 IST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000064s latency).
Not shown: 65528 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
631/tcp   open  ipp
3306/tcp  open  mysql
8501/tcp  open  cmtp-mgt
11434/tcp open  unknown
33060/tcp open  mysqlx

Nmap done: 1 IP address (1 host up) scanned in 0.92 seconds
```

Figure 4: Nmap Full Port Scan Results

The full port scan identifies open ports and active services running on the target system.

Step 3 – Stop and Save Captured Packets

After traffic generation, the captured packets are saved in .pcapng format for offline analysis.

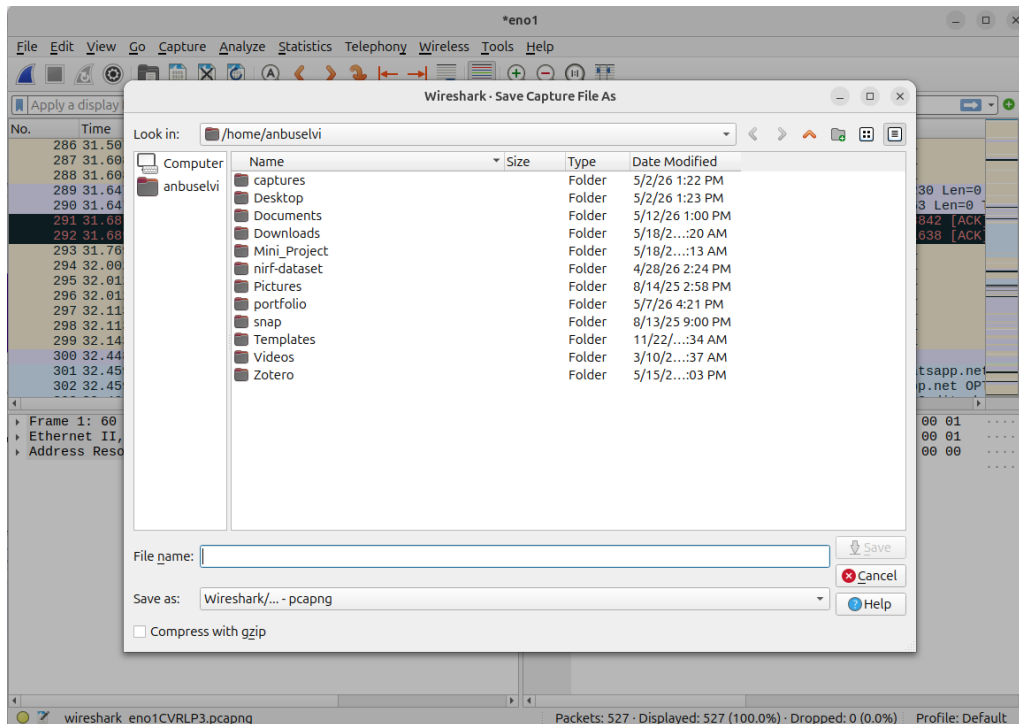


Figure 5: Saving Captured Packets as PCAPNG File

Wireshark provides options to save captured traffic files for future packet analysis and feature extraction.

Step 4 – Export Features

tshark extracts packet-level features from the capture file and stores them in CSV format.

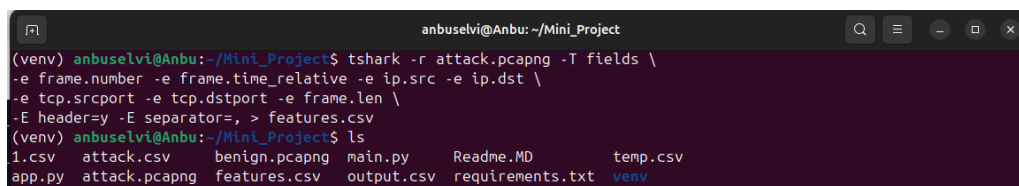


Figure 6: tshark Feature Extraction Command Execution

Packet attributes such as source IP, destination IP, ports, and packet length are exported into a CSV dataset.

Step 5 – Run Detection Dashboard

The Streamlit application is executed to launch the anomaly detection dashboard.

```
anbuselvi@Anbu: ~/Mini_Project
app.py attack.pcapng features.csv output.csv requirements.txt venv
(venv) anbuselvi@Anbu:~/Mini_Project$ streamlit run app.py
2026-05-18 10:22:39.800 Unicorn server started on 0.0.0.0:8502

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8502
Network URL: http://10.10.0.106:8502

2026-05-18 10:22:42.228 Please replace 'use_container_width' with 'width'.
'use_container_width' will be removed after 2025-12-31.
For 'use_container_width=True', use 'width='stretch''. For 'use_container_width=False', use 'width='content''.
2026-05-18 10:22:42.281 Please replace 'use_container_width' with 'width'.
'use_container_width' will be removed after 2025-12-31.
For 'use_container_width=True', use 'width='stretch''. For 'use_container_width=False', use 'width='content''.
2026-05-18 10:22:42.283 Please replace 'use_container_width' with 'width'.
'u...'
```

Figure 7: Running Streamlit Detection Dashboard

The Streamlit server initializes the real-time dashboard for network traffic visualization and anomaly monitoring.

Step 6 – Observe Dashboard Results

The dashboard visualizes anomalies, suspicious hosts, packet statistics, and traffic behavior.

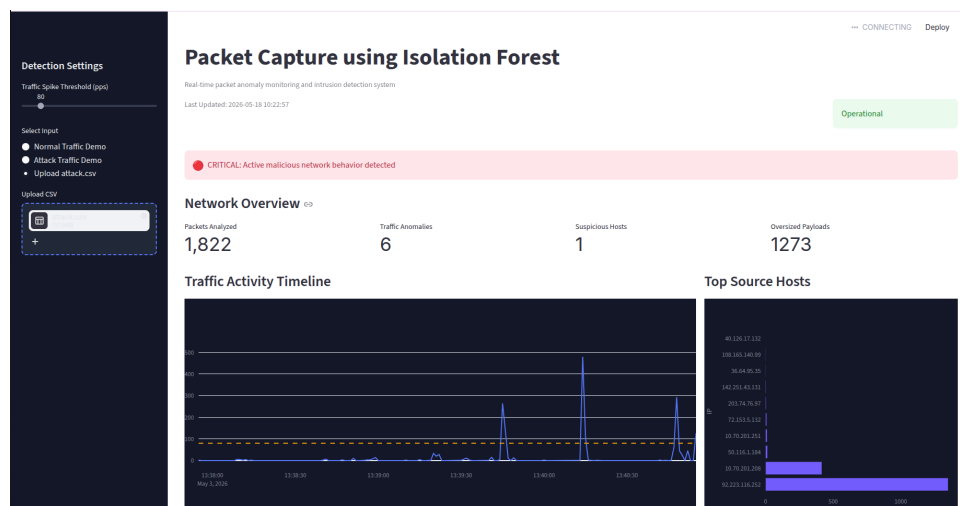


Figure 8: ML-Based Threat Analysis Dashboard

The dashboard identifies anomalous hosts and oversized payload events using Isolation Forest.

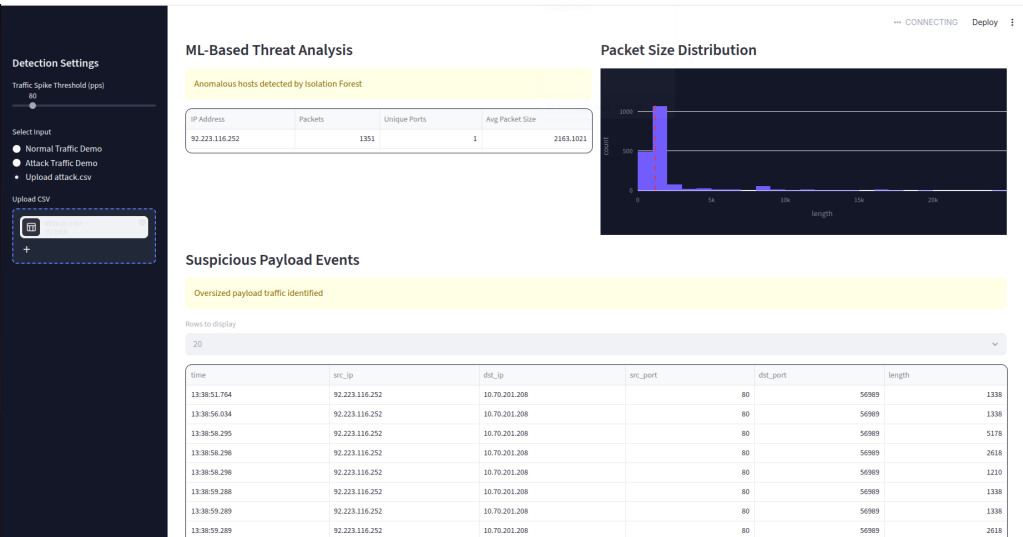


Figure 9: Real-Time Network Traffic Monitoring Dashboard

Traffic activity timelines and top source hosts are displayed for continuous monitoring.

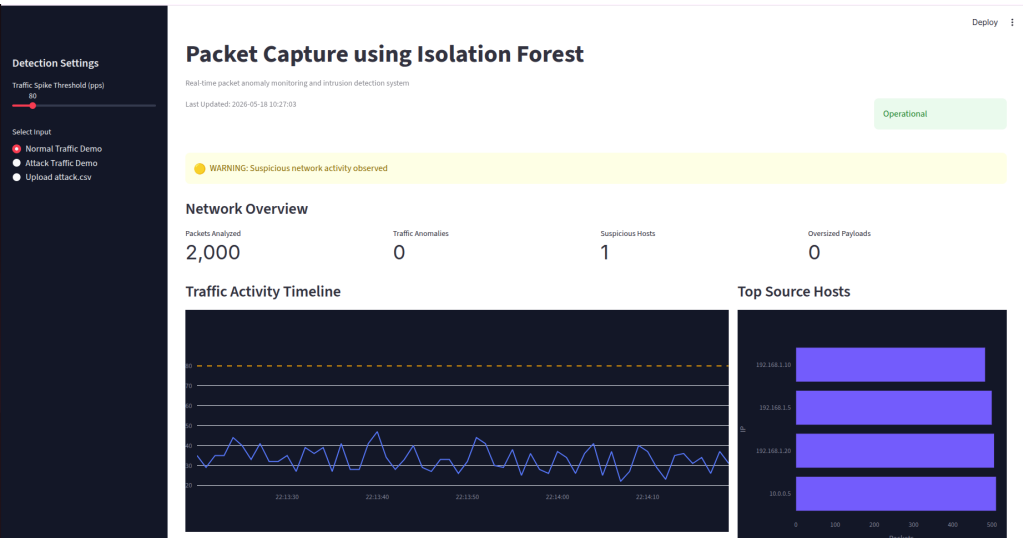


Figure 10: Normal Traffic Detection Results

The dashboard displays normal traffic behavior with minimal anomalies detected.

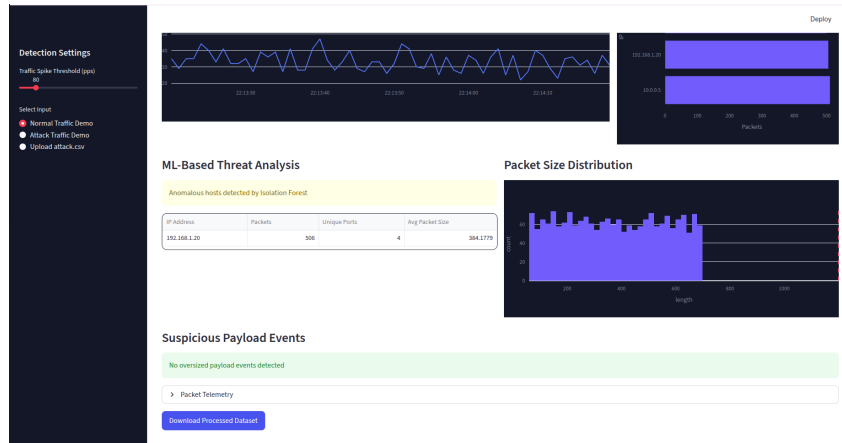


Figure 11: Packet Size Distribution and Threat Analysis

Packet size distribution graphs help visualize suspicious packet payload activity and abnormal traffic behavior.

Step 7 – Downloaded CSV File

The processed packet-level traffic dataset is exported and stored in CSV format.

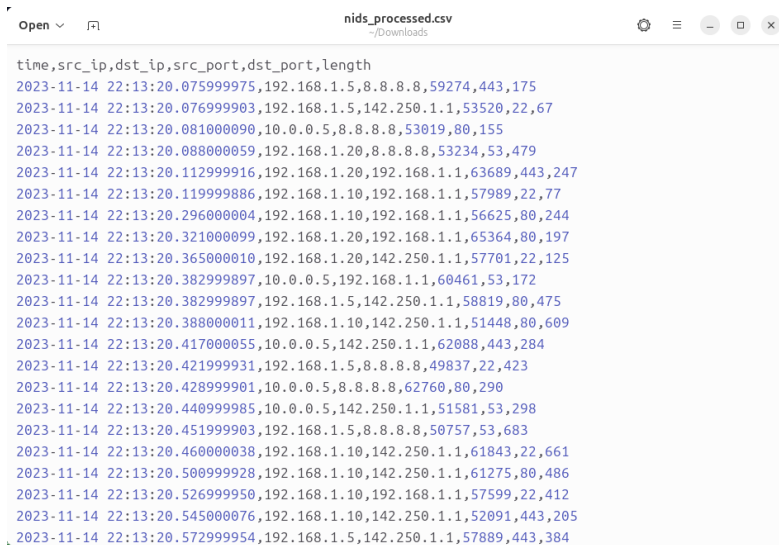


Figure 12: Extracted Packet Features in CSV Format

The CSV file contains extracted packet features used for anomaly detection and traffic analysis.

9. Conclusion

This project successfully demonstrated packet-level anomaly detection using Wireshark, tshark, Machine Learning, and Streamlit. Packet features were extracted and analyzed to identify abnormal traffic behavior such as traffic spikes and oversized packets. Isolation Forest enabled ML-based anomaly detection, while Streamlit provided real-time visualization and monitoring. The workflow provides a scalable and low-cost solution for network traffic analysis and cybersecurity monitoring.